



ADVENTIST UNIVERSITY OF CENTRAL AFRICA
Faculty of Information Technology

RwandaRide

A Real-Time Ride-Monitoring Data Pipeline
with Machine-Learning ETA Prediction

Apache Hadoop HDFS • Apache Kafka • Apache Spark & MLlib • Django

Big Data Essentials | Group Final Exam Project

Group Members

101420	Patrick Niyonshuti
101414	Ernest Uwimpuhwe
101366	Gisele Uwibereyeho

Lecturer

Dr Kundan Kumar

Kigali, Rwanda • September 2026

1 Problem Statement

Ride-hailing dispatch is a decision problem under time pressure. A dispatcher watching a live fleet needs two things: to know which trips are running materially behind so a rider can be called, and to know where and when demand is about to concentrate so drivers can be repositioned before the surge rather than after it. Neither question can be answered from a single database row — the first needs every event as it arrives, the second needs the entire history aggregated. That is why this problem requires the architecture it has.

In one sentence.

The Kafka stream carries trip events as they happen, the Spark MLlib model predicts trip duration in minutes, and the dashboard shows dispatchers which trips are running late and which zones are short on drivers.

2.39 M

trip events carried end to end,
every stage

60 drivers

across 8 Kigali zones, fares in
RWF

2.15 min

mean ETA error of the deployed
model

The domain is modelled on Kigali — eight zones (Kacyiru, Nyamirambo, Remera, Kimironko, Gikondo, Kicukiro, Nyarugenge, Gisozi), sixty drivers, fares in Rwandan francs. The case study is listed in Section 4 of the project guidelines.

1.1 Why this needs big data tools rather than a web app and a database

Three specific failures of the single-server model, each addressed by one component of the stack.

Volume HDFS

2.39 M events retained in full as date-partitioned Parquet. A full-history scan at that size would lock a relational table for seconds; HDFS is read in parallel, one Spark task per block.

Velocity Kafka

The generator's rate is an environment variable, changed live, with the effect visible in Kafka offsets, MariaDB rows and HDFS directory size simultaneously.

Variety Schema envelope

Identifiers, continuous measures, categoricals and a timestamp in one self-describing record — which is what lets Kafka Connect create tables and write columnar Parquet with no hand-written DDL.

2 Architecture of the System Built

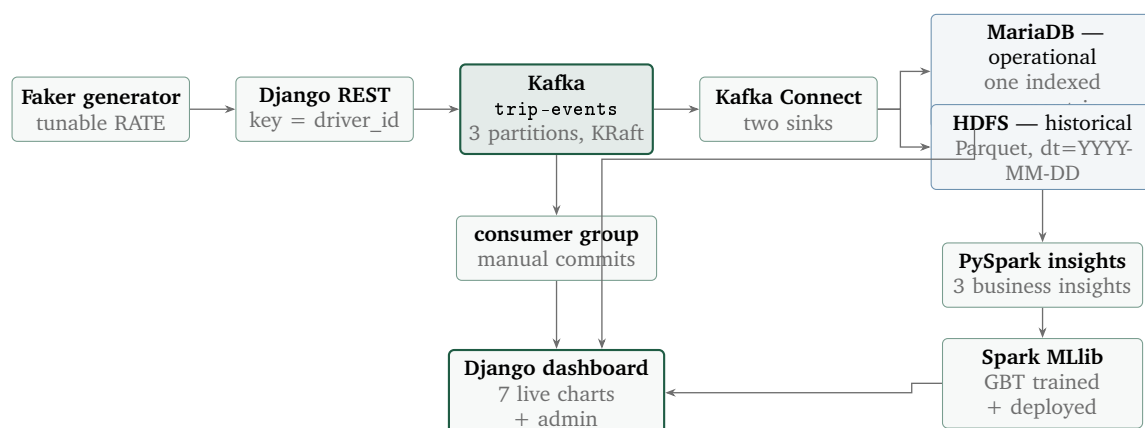


Figure 1: The system as built. One Kafka topic is read three ways by three independent consumers — that fan-out is why the producer never writes twice, and why one sink can fail without affecting the others. No stage copies data by hand.

Layer	Technology	Why this one
Generation	Python + Faker	Tunable rate; the label comes from a known noisy function, so model quality can be judged against a theoretical ceiling.
Producer	Django REST Framework	Validates at the edge, so malformed records never reach Kafka; returns the partition and offset each record landed on.
Streaming	Kafka 3.9 (KRaft)	No ZooKeeper to operate; three partitions allow a multi-consumer group.
Ingestion	Kafka Connect	Declarative sinks, no bespoke ETL code.
Operational store	MariaDB	Indexed current-state reads for the dashboard.
Historical store	HDFS 3.2 + Parquet	Replicated block storage, columnar and date-partitioned for pruning.
Processing	Spark 4.2 / PySpark	Parallel reads from HDFS, one task per block.
Machine learning	Spark MLlib GBT	Captures the multiplicative structure of the data.
Serving	Django 4.2 LTS	Dashboard plus admin over the same schema.

Two version constraints shaped the build. Django 4.2 LTS was required because 5.x needs MariaDB 10.5+ and 6.x needs 10.6. Kafka Connect requires Java 17 — the HDFS3 connector bundles Spring classes compiled for it — while Hadoop runs on Java 8, so `JAVA_HOME` is overridden for the Connect worker only.

2.1 The journey of one trip event

The components are easiest to understand from the point of view of a single record. Driver D0042's 12.4 km run from Remera to Gisozi is invented by `generate.py`, validated field by field by DRF's `TripSerializer` — malformed records return HTTP 400 and never reach Kafka — then hashed on its key to partition 1, offset 280,531, where it becomes durable on disk. From there three independent consumers read it without coordinating: the JDBC sink upserts it into MariaDB, the HDFS sink appends it to Parquet under `dt=2026-09-18/`, and our consumer group compares 38.2 actual minutes against 24.8 expected — $1.54\times$, under the $3.0\times$ threshold, so no alert. It is later aggregated by `insights.py`, learned from by `train_model.py`, scored by `score_batch.py`, and finally drawn on the dispatch board. Every one of those hand-offs is between two separate processes.

2.2 Kafka: partitioning, offsets and delivery guarantees

The message key is `driver_id`. Per-driver ordering is what the domain needs — a driver cannot start trip 2 before finishing trip 1 — while global ordering is not required, which is what makes three partitions safe. Keying on `trip_id` would distribute perfectly but give no useful ordering since every trip is unique; publishing without a key would round-robin and lose ordering entirely. The measured cost is skew: with sixty keys over three partitions, partition 1 carried roughly twice the traffic. Skew is the price of the ordering guarantee — any scheme preserving same-key-same-partition can be unbalanced by the key distribution.

Offsets are committed manually: `enable_auto_commit=False`, and per batch the sequence is process → write to MariaDB → commit the database transaction → commit Kafka offsets. The Connect HDFS sink, by contrast, is genuinely exactly-once — it names files by offset range and recovers partial writes from a write-ahead log. Two different guarantees in one system, each chosen for a reason rather than by default.

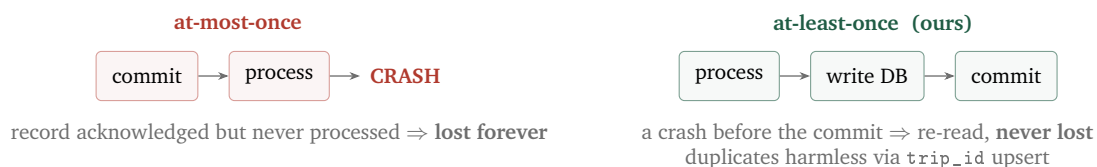


Figure 2: What a crash costs, and why the ordering of the commit matters. Writing to the database before committing the offset is what converts a lost record into a harmless duplicate.

2.3 Resilience, observed three times

Three unplanned failures tested the design, and in every case the durable log did what a durable log exists for.

Failure	What broke	Outcome
MariaDB unavailable after a machine restart	The JDBC sink failed validation and the Connect worker exited; Kafka kept accepting records.	On restart the sink resumed from its committed offsets and drained ~15,000 records — no loss, no duplication.
Power loss left an unclosed HDFS lease	The NameNode could not report the file's block length, so the connector's recovery log was unreadable and that task was killed — while the database sink continued, because the sinks are independent consumer groups.	Kafka retained all 1,045,459 records the sink missed; <code>hdfs debug recoverLease</code> closed the lease and the sink replayed the entire backlog.
The operational store lost entirely	A MariaDB service change left the <code>ridehail</code> database absent.	Schema recreated, the sink's consumer group reset to the earliest offset, and the full 2.39-million-row database rebuilt from the log with no manual data copying.

Table 1: Log-based replay is the strongest practical argument for placing a durable log at the centre of an architecture — and none of these three incidents was planned.

3 Data Schema

Field	Type	Role	Notes
<code>trip_id</code>	string	key	UUID; primary key in MariaDB, which is what makes the sink idempotent.
<code>driver_id</code>	string	partition key	D0001–D0060, a fixed pool so partitioning is demonstrable.
<code>rider_id</code>	string	—	
<code>pickup_zone</code>	string	feature	One of eight Kigali zones.
<code>dropoff_zone</code>	string	feature	
<code>distance_km</code>	double	feature	1.0–28.0
<code>pickup_hour</code>	int	feature	Drawn from a commuter distribution.
<code>day_of_week</code>	int	feature	1–7, Friday weighted highest.
<code>weather</code>	string	feature	clear / cloudy / rain / heavy_rain
<code>traffic_level</code>	string	feature	light / moderate / heavy / gridlock
<code>surge_multiplier</code>	double	feature	1.0–2.0
<code>fare_rwf</code>	double	<i>excluded</i>	Derived from the target — see §4.2.
<code>duration_min</code>	double	target	Minutes.
<code>event_ts</code>	string	—	ISO 8601, UTC.

Table 2: The trip event. Eight of fourteen fields reach the model; one is the target and one is deliberately withheld.

How the label is generated, which is what makes evaluation meaningful:

$$\text{duration} = \frac{\text{distance}}{30 \text{ km/h}} \times 60 \times f_{\text{traffic}} \times f_{\text{weather}} \times f_{\text{rush}} + \mathcal{N}(0, 1.8)$$

with $f_{\text{traffic}} \in \{1.0, 1.25, 1.6, 2.2\}$, $f_{\text{weather}} \in \{1.0, 1.03, 1.18, 1.35\}$ and $f_{\text{rush}} = 1.2$ in hours 07–08 and 17–19.

Two consequences matter. The relationship is **multiplicative**, so the factors interact with distance — which dictates the model choice in §4. And the Gaussian term is **irreducible**: no model can predict it, so RMSE cannot fall far below 1.8 minutes and R^2 cannot reach 1.0. That is an explicit ceiling against which to judge the result.

3.1 The two stores, and why the data is deliberately duplicated

	MariaDB — operational	HDFS — historical
Answers	What is true <i>right now</i> , in milliseconds	What <i>happened</i> , by parallel scan
Layout	One indexed row per trip, updatable	Append-only, columnar Parquet, date-partitioned
Resilience	Single server	Same block replicated across DataNodes; survives node loss
Scales by	Buying a bigger machine	Adding machines

One store optimised for both would be good at neither — which is why the same trip is written twice, by two independent sinks, from one Kafka topic.

Storage layout. HDFS writes `/data/trip-events/dt=2026-09-18/trip-events+0+0000020150+0000020349.parquet`: the filename carries the Kafka partition and the exact offset range, which is how the connector achieves exactly-once delivery into HDFS — on restart it knows precisely which offsets are already durable.

Date partitioning means a query filtered on `dt` reads one directory rather than the whole history — partition pruning — and Parquet's columnar layout means a job needing four of fourteen columns reads only those four. In practice Spark read 7.6 MB to aggregate 450,000 rows, and the same data occupied 172.6 MB as Parquet against 517.7 MB as text.

4 Model Choice and Evaluation

Predicting `duration_min` is supervised regression — the target is a continuous number and every training row has a known correct answer. Every row entered through the DRF producer, was carried by Kafka, was written to HDFS by Kafka Connect, and was read back out by Spark. No external dataset was used.

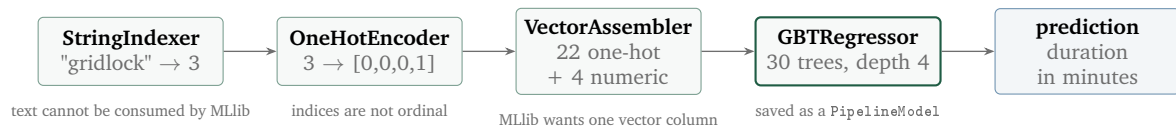


Figure 3: The feature pipeline. Each stage exists for a reason. Bundling them into a Pipeline is what prevents training-time and scoring-time encoding from drifting apart.

4.1 Why gradient-boosted trees rather than linear regression

The data is multiplicative: traffic, weather and rush hour scale base travel time and therefore interact with distance — heavy traffic costs more absolute minutes on a 20 km trip than on a 2 km one. A linear model can only sum contributions, so it cannot represent *traffic multiplies the effect of distance* without hand-built interaction terms. A decision tree splits on distance and then splits again on traffic within that branch, which is an interaction obtained without being told. Gradient boosting then builds thirty such shallow trees in sequence, each correcting the residuals of the last. To settle the claim by measurement rather than argument, a `LinearRegression` baseline was trained on the identical split in the same job.

4.2 Target leakage — the feature we deliberately withheld

`fare_rwf` is computed from `duration_min`: `fare = 500 + distance × 420 × surge + duration × 25`. A model given the fare could rearrange that equation and recover the answer, scoring almost perfectly — and be useless, because the fare is not known until the trip ends, and the whole point of an ETA is to have it before. Excluding it is a deliberate design decision, not an oversight.

4.3 Evaluation

Model	RMSE (min)	MAE (min)	R^2
GBTRegressor (30 trees, depth 4)	2.821	2.151	0.991
LinearRegression (baseline, same split)	8.755	6.478	0.911

Trained on 58,048 rows; tested on 14,316 never seen during training. Theoretical floor 1.8 min.

Gradient boosting reduced RMSE by 67.8%.

RMSE, MAE and R^2 are the appropriate metrics for regression; accuracy, precision, recall and F1 do not apply to a continuous target. **MAE of 2.151 minutes** is the figure a dispatcher understands — the average number of minutes the ETA is wrong. RMSE is higher because it squares errors before averaging, penalising the occasional large miss, which matters more when an ETA has been promised to a rider. R^2 of 0.991 means the features explain 99.1% of the variance in duration.

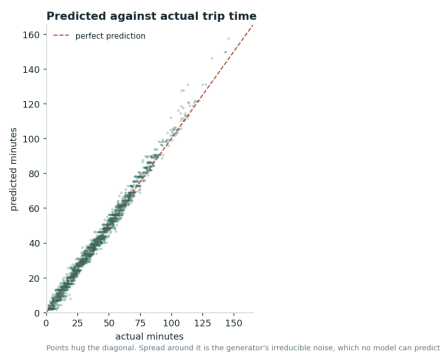
The ceiling is the most important interpretive point.

Because the generator adds $\mathcal{N}(0, 1.8)$ to every duration, an RMSE near 1.8 is the best *any* model could reach. At 2.821 the model has captured very nearly all the learnable signal — and a result substantially *below* 1.8 would have indicated leakage rather than skill, which is precisely why §4.2 matters.

Feature importances place `distance_km` at 0.60, far ahead of `pickup_hour` at 0.096 and the traffic encodings; that physics dominates a travel-time model is a sanity check passed.

Deployment. `score_batch.py` is a separate job in a separate process: it loads the persisted model from `hdfs://localhost:9000/models/eta_gbt` and scores recent trips it has never seen, writing actual-versus-predicted to MariaDB for the dashboard. Nothing about training is repeated — the fitted indexers, encoders and trees all return from disk exactly as saved.

Why a 20% sample. A first attempt at 60 trees of depth 5 over 359,764 rows exceeded the driver's heartbeat timeout — Spark logged no recent heartbeats: 132829 ms and killed its own executor, because boosting is sequential and cannot parallelise across iterations. The reduced configuration was *validated* rather than assumed: R^2 of 0.991 against the noise ceiling means the sample carried the full learnable signal.



(a) 2,000 scored trips against truth.

```
=== sample predictions vs actual (test set) ===
```

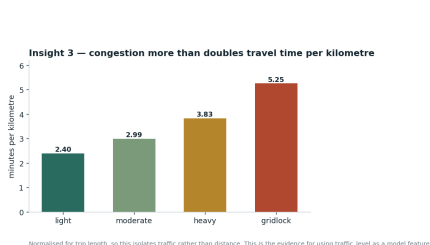
km	traffic_level	weather	pickup_hour	actual_min	predicted_min	error
18.5	gridlock	clear	8	96.05	98.49	2.44
3.27	heavy	clear	8	10.62	12.65	2.03
3.77	heavy	clear	8	18.71	12.65	-6.06
15.55	heavy	clear	8	58.39	63.27	4.88
3.16	light	clear	8	4.75	7.21	2.46
8.29	light	clear	8	21.79	20.25	-1.54
14.96	light	clear	18	40.04	36.13	-3.91
1.81	moderate	clear	19	5.28	6.87	1.59
8.8	moderate	clear	8	23.05	27.41	4.36
13.6	moderate	clear	6	33.06	35.68	2.62
14.38	moderate	clear	7	39.12	45.56	6.44
22.2	moderate	clear	19	65.84	67.45	1.61

(b) Real output from the deployment job.

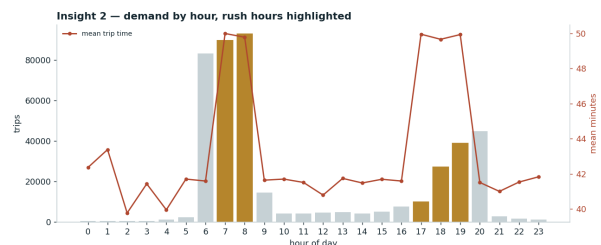
Figure 4: Left: points hug the diagonal across the whole range; the spread is the generator's irreducible noise. Right: the 18.5 km gridlock trip is predicted within 2.4 minutes, while the 3.27 km trip is off by 2.03 — on a short trip the ± 1.8 -minute noise is proportionally much larger. The largest errors cluster on long gridlock trips, consistent with gridlock being the most thinly sampled traffic category.

5 Insights Found

Three PySpark batch jobs read Parquet directly from HDFS, deduplicate on `trip_id` (HDFS is append-only, so a Connect restart can rewrite a record — deduplication is what keeps the analytics correct despite at-least-once delivery upstream), apply range filters, and write results back to MariaDB for the dashboard.



(a) Minutes per kilometre by traffic level, normalised for trip length.



(b) Trips by hour, rush hours in amber, with mean duration on the right axis.

Figure 5: The two quantitative insights. Both are computed by PySpark over the full history in HDFS.

Insight 1 — demand is bimodal. Peaks at 07:00–08:00 and 17:00–19:00. More usefully, mean duration rises to roughly 50 minutes in the rush hours against roughly 41 elsewhere: the $1.2\times$ congestion effect surfacing in aggregate, and what gives `pickup_hour` genuine predictive value for the model.

This insight also produced the project’s most instructive error. The first generator stamped `pickup_hour` from the wall clock, so every record carried whatever hour the generator happened to be running — a clean, plausible chart that was entirely wrong. After correcting the generator, training filters on the HDFS date partition holding clean records, which *also* exercises partition pruning since Spark skips the older directory entirely rather than reading and discarding it. Records written before the fix retain the artefact, so counts on hours 6–8 and 19–20 remain inflated; the duration line is unaffected.

Insight 2 — congestion more than doubles travel time per kilometre, the clearest and most stable result in the study:

Traffic	Trips	Mean minutes	Min/km (PySpark)	Min/km (MapReduce)
gridlock	148,066	74.73	5.146	5.146
heavy	442,376	54.31	3.745	3.745
moderate	699,467	42.50	2.929	2.929
light	552,074	33.93	2.346	2.346

Table 3: Two independent implementations agreeing to three decimal places, with counts summing to exactly 1,841,983 — evidence the result itself is correct rather than an artefact of one framework.

Normalising by distance is what makes this meaningful; without it the metric would mostly restate that longer trips take longer. The monotonic gradient from 2.346 to 5.146 min/km is the direct evidence for including `traffic_level` as a model feature, and it held stable as the dataset grew twenty-fold.

Insight 3 — demand is near-uniform across zones, busiest to quietest separated by under 2%. This is a negative result and is reported as one: the generator samples zones uniformly, so the flatness confirms the pipeline is faithful rather than revealing anything about Kigali. Its operational reading is that dispatch gains come from *timing* rather than *geography* — exactly what Insights 1 and 2 quantify.

5.1 The dashboard

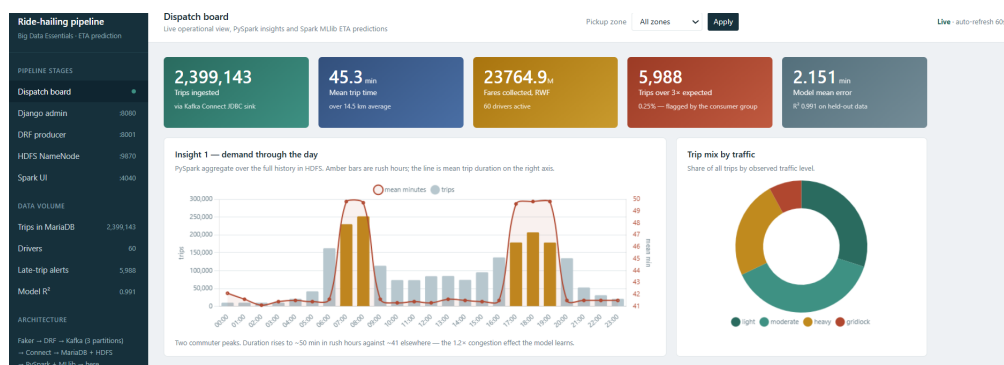


Figure 6: The dispatch board. Live operational counts from MariaDB via the Kafka Connect JDBC sink, PySpark insights from HDFS, and MLlib model error — with seven charts recomputed from query results on every request, so they move with the data rather than freezing at generation time.

Two Django applications serve the same schema: the dispatch board on port 8000, and Django admin on 8080 using `managed = False` models over the existing tables. That declaration is the important one — Django is a *reader* here, while Kafka Connect, the consumer group and the Spark jobs own the writes. Each `ModelAdmin` is read-only, because permitting an edit would let the operational store drift from the HDFS event history that is the actual source of truth.

Four enhancements beyond the reference architecture: (1) late-trip alerts from the consumer group’s business logic, each shown with *the Kafka partition and offset that produced it*; (2) a pickup-zone drill-down that re-scopes the operational panels while leaving HDFS-derived insights global; (3) role-based access, with raw records behind a login and aggregates open; and (4) prediction-error analysis, which revealed that the largest errors cluster on long gridlock trips.

The alert threshold was itself tuned empirically — at $1.5\times$ expected duration roughly 60% of trips were flagged, because gridlock alone is a $2.2\times$ multiplier, so the rule was mostly detecting ordinary congestion. At $3.0\times$ the rate falls to a fraction of a percent.

6 Limitations and Future Work

The system runs on a single machine, which bounds it in four ways — each with a known remedy rather than an open question.

Single-node deployment. One broker at replication factor 1 and one DataNode mean fault tolerance is configured but not demonstrated across physical machines. HDFS already declares replication factor 3 and the topic is already partitioned, so the production path is adding nodes rather than changing code: HDFS replicates under-replicated blocks automatically, and a three-broker cluster at RF 3 would survive a broker loss with no application change.

File size, tuned for demonstration. `flush.size=200` and `rotate.interval.ms=60000` produce ~ 18 KB files against a 256 MB block so that output appears within a demonstration window. At production settings the same connector writes block-sized files — these are configuration parameters, not architectural properties.

Training on a 20% sample. Bounded by single-executor memory rather than by data availability, and validated rather than assumed: R^2 of 0.991 against a noise floor of 1.8 minutes shows the sample carried the full learnable signal. On a multi-executor cluster the constraint does not arise.

Synthetic source data. The pipeline is real end to end; the domain conclusions inherit the generator's assumptions. This was a deliberate trade — a known data-generating function is precisely what makes the noise ceiling in §4.3 computable, and therefore what allows the model's accuracy to be judged as good rather than merely reported as low. Against a production feed the same pipeline runs unchanged, but without that reference point.

Production deployment would additionally require a WSGI server, credentialed database access and TLS between components — standard hardening, outside the scope of a pipeline demonstration.

Conclusion

The system ingests trip events at a configurable rate, carries them through Kafka with a justified partitioning strategy and explicitly managed offsets, lands them automatically in both an operational relational store and a replicated columnar historical store, processes that history at scale with PySpark, trains and deploys an MLlib model that predicts trip duration to within roughly two minutes, and surfaces all of it in a live Django dashboard. Over 2.39 million events passed through it end to end.

Three findings worth carrying away

- Congestion more than doubles travel time per kilometre, from 2.346 to 5.146 min/km.
- Gradient boosting reduced prediction error by 67.8% against a linear baseline, because the underlying relationship is multiplicative and trees capture interactions that additive models cannot.
- The model's accuracy sits close to a theoretical floor set by noise deliberately built into the data — which is the only reason we can say the result is *good* rather than merely *low*.

The most instructive parts of the work, however, were not components but failures. Three separate incidents — a database outage, an unclosed HDFS lease after a power loss, and the complete loss of the operational store — were each survived without data loss, because Kafka retained everything and every consumer resumed from its committed offsets. And one bug taught more than any of them: an hour field stamped from the wall clock rather than simulated, which produced a plausible-looking chart that was entirely wrong. Finding it required questioning our own output rather than trusting it — which is what distinguishes a pipeline that runs from one that can be believed.